# Chapter 4

# Database Recovery Techniques

# Chapter Outline

- Database Recovery
- Backup and Recovery Concepts
- Transactions and Recovery
- Recovery Techniques
- Recovery Concepts Based on Deferred Update
- Recovery Concepts Based on Immediate Update
- Shadow Paging
- The ARIES Recovery Algorithm
- Recovery in Multi database Systems

# Database Recovery

- Database recovery is the process of restoring database to a correct state in the event of a failure.
- ***DR is the process of eliminating the effects of a failure from the database*.**
- ***Recovery***, in database systems terminology, is called ***restoring the last consistent state of the data items***.

➢ Need for Recovery Control

- Two types of storage: volatile (main memory) and nonvolatile.
- Volatile storage does not survive system crashes.
- Stable storage represents information that has been replicated in several nonvolatile storage media with independent failure modes.

# Types of failures

- A failure is a state where data inconsistency is visible to transactions if they are scheduled for execution.
- The kind of failure could be:
  - System crashes, resulting in loss of main memory.
  - Media failures, resulting in loss of parts of secondary storage.
  - Application software errors.
  - Natural physical disasters.
  - Carelessness or unintentional destruction of data or facilities.
  - Sabotage.

# …CON'T

- In databases usually a failure can generally be categorized as one of the following major groups:
  - **1)Transaction failure**: a transaction cannot continue with its execution, therefore, it is aborted and if desired it may be restarted at some other time.
    - Reasons: Deadlock, timeout, protection violation, or system error.
  - **2)System failure:** the database system is unable to process any transactions.
    - Some of the common reasons of system failure are:
      - wrong data input, register overflow, addressing error, power failure, memory failure, etc.
  - **3)Media failure**: failure of non-volatile storage media (mainly disk).
    - Some of the common reasons are:
      - head crash, dust on the recording surfaces, fire, etc.

# …CON'T

- To make the database secured, one should formulate a "plan of attack" in advance.
- The plan will be used in case of database insecurity that may range from minor inconsistency to total loss of the data due to hazardous events.
- The basic steps in performing a recovery are
  - 1) Isolating the database from other users. Occasionally, you may need to drop and re-create the database to continue the recovery.
  - 2) Restoring the database from the most recent useable dump.
  - 3) Applying transaction log dumps, in the correct sequence, to the database to make the data as current as possible.

# …CON'T

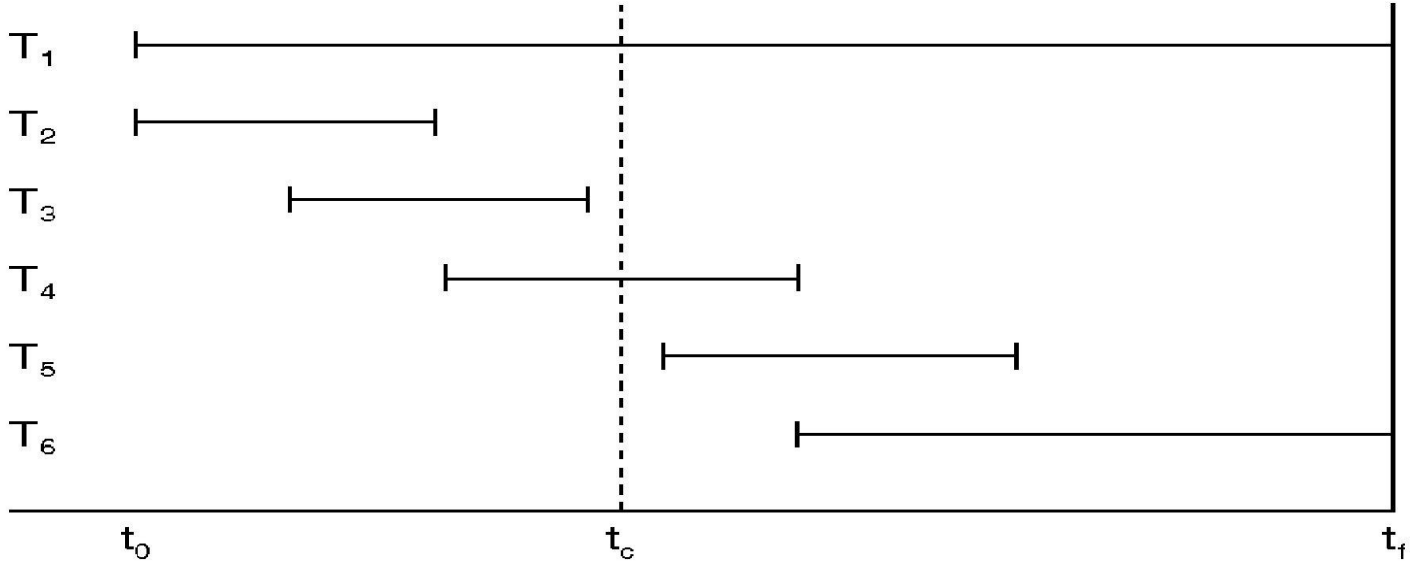
- It is a good idea to test your backup and recovery plans periodically by loading the backups and transaction logs into a test database and verifying that your procedure really works.
- One can recover databases after three basic types of problems: user error, software failure, and hardware failure.
- Each type of failure requires a recovery mechanism.
- In a transaction recovery, the effect of failed transaction is removed from the database, if any.
- In a system failure, the effects of failed transactions have to be removed from the database and the effects of *completed* transactions have to be *installed* in the database.
- The database recovery manger is responsible to guarantee the atomicity and durability properties of the ACID property.

# Transactions and Recovery

- Transactions represent basic unit of recovery.
- Recovery manager responsible for atomicity and durability.
- If failure occurs between commit and database buffers being flushed to secondary storage then, to ensure durability, recovery manager has to *redo* (*roll forward*) transaction's updates.
- If transaction had not committed at failure time, recovery manager has to *undo* (*rollback*) any effects of that transaction for atomicity.
- Partial undo - only one transaction has to be undone.
- Global undo - all transactions have to be undone.

# …CON'T

- **Transaction Log**: Execution history of concurrent transactions.




- DBMS starts at time to, but fails at time tf.
  - Assume data for transactions T2 and T3 have been written to secondary storage.
- T1 and T6 have to be undone. In absence of any other information, recovery manager has to redo T2, T3, T4, and T5.
- tc is the checkpoint time by the DBMS

# Recovery Facilities

- DBMS should provide following facilities to assist with recovery:
  - **Backup mechanism:** that makes periodic backup copies of database.
  - **Logging facility:** that keeps track of current state of transactions and database changes.
  - **Checkpoint facility:** that enables updates to database in progress to be made permanent.
  - **Recovery manger:** This allows the DBMS to restore the database to a consistent state following a failure.
- Restoring the database means transforming the state of the database to the immediate good state before the failure.
- To do this, the change made on the database should be preserved.
- Such kind of information is stored in a ***system log*** or ***transaction log*** file.

# Log File

- Contains information about all updates to database:
  - Transaction records.
  - Checkpoint records.
- Often used for other purposes (for example, auditing).
- Transaction records contain:
  - Transaction identifier.
  - Type of log record, (transaction start, insert, update, delete, abort, commit).
  - Identifier of data item affected by database action (insert, delete, and update operations).
  - Before-image of data item.
  - After-image of data item.
- Log management information.
- Log file sometimes split into two separate random-access files.
- Potential bottleneck; critical in determining overall performance.

# …CON'T

| *Tid* | *Time* | *Operation* | *Object* | *Before image* | *After image* | *PPtr* | *NPtr* |
|---|---|---|---|---|---|---|---|
| T1 | 10:12 | START | | | | 0 | 2 |
| T1 | 10:13 | UPDATE | STAFF SL21 | (old value) | (new value) | 1 | 8 |
| T2 | 10:14 | START | | | | 0 | 4 |
| T2 | 10:16 | INSERT | STAFF SG37 | | (new value) | 3 | 5 |
| T2 | 10:17 | DELETE | STAFF SA9 | (old value) | | 4 | 6 |
| T2 | 10:17 | UPDATE | PROPERTY PG16 | (old value) | (new value) | 5 | 9 |
| T3 | 10:18 | START | | | | 0 | 11 |
| T1 | 10:18 | COMMIT | | | | 2 | 0 |
| | 10:19 | CHECKPOINT | T2, T3 | | | | |
| T2 | 10:19 | COMMIT | | | | 6 | 0 |
| T3 | 10:20 | INSERT | PROPERTY PG4 | | (new value) | 7 | 12 |
| T3 | 10:21 | COMMIT | | | | 11 | 0 |

# Check pointing

- Checkpoint: is a point of synchronization between database and log file. All buffers are force-written to secondary storage.
  - Checkpoint record is created containing identifiers of all active transactions.
  - When failure occurs, redo all transactions that committed since the checkpoint and undo all transactions active at time of crash.
  - In previous example, with checkpoint at time tc, changes made by T2 and T3 have been written to secondary storage.
  - Thus:
    - Only redo T4 and T5, Undo transactions T1 and T6.

# Recovery Techniques

- Damage to the database could be either physical and relate which will result in the loss of the data stored or just inconsistency of the database state after the failure.

- For each we can have a recover mechanism:
  - 1) If database has been damaged:
    - Need to restore last backup copy of database and reapply updates of committed transactions using log file.
    - Extensive damage/catastrophic failure: physical media failure; is restored by using the backup copy and by re executing the committed transactions from the log up to the time of failure.

# …CON'T

- 2) ) If database is only inconsistent:
  - No physical damage/only inconsistent: the restoring is done by reversing the changes made on the database by consulting the transaction log.
  - Need to undo changes that caused inconsistency. May also need to redo some transactions to ensure updates reach secondary storage.
  - Do not need backup, but can restore database using before- and after-images in the log file.

# Recovery Techniques for Inconsistent Database State

- **Recovery is required if only the database is updated.**
- **The kind of recovery also depends on the kind of update made on the database.**
- **Database update:** A transaction's updates to the database can be applied in two ways:
  - Three main recovery techniques:
    - Deferred Update
    - Immediate Update
    - Shadow Paging

# Deferred Update

- Updates are not written to the database until after a transaction has reached its commit point.
- If transaction fails before commit, it will not have modified database and so **no undoing** of changes required.
- May be necessary to **redo** updates of committed transactions as their effect may not have reached database.
- If a transaction aborts, ignore the log record for it. And do nothing with transaction having a "transaction start" and "Transaction abort" log records
- A transaction first modifies all its data items and then writes all its updates to the final copy of the database. No change is going to be recorded on the database before commit. The changes will be made only on the local transaction workplace. Update on the actual database is made after commit and after the change is recorded on the log. Since there is no need to perform undo operation it is also called **NO-UNDO/REDO Algorithm**
- The redo operations are made in the order they were written to log.

# Immediate Update/ Update-In-Place

- Updates are applied to database as they occur.
- Need to redo updates of committed transactions following a failure.
- May need to undo effects of transactions that had not committed at time of failure.
- Essential that log records are written before write to database. **Write ahead log protocol**.
- If no "transaction commit" record in log, then that transaction was active at failure and must be **undone**.
- Undo operations are performed in reverse order in which they were written to log.

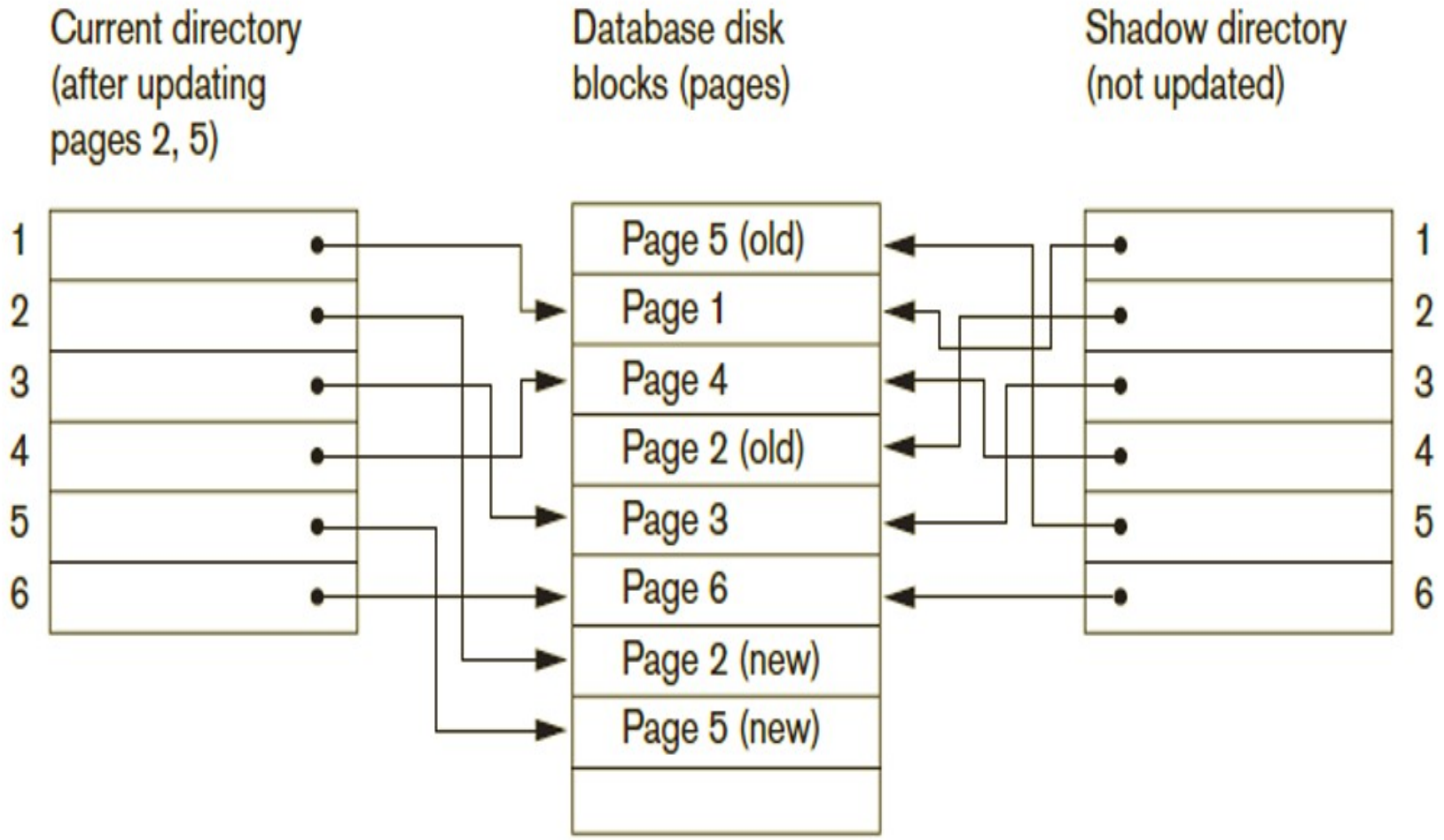
# …CON'T

- As soon as a transaction updates a data item, it updates the final copy of the database on the database disk.
- During making the update, the change will be recorded on the transaction log to permit rollback operation in case of failure.
- UNDO and REDO are required to make the transaction consistent.
  - Thus it is called **UNDO/REDO Algorithm.**
  - This algorithm will undo all updates made in place before commit.
  - The redo is required because some operations which are completed but not committed should go to the database.
  - If we don't have the second scenario, then the other variation of this algorithm is called **UNDO/NO-REDO Algorithm.**

# Shadow Paging

- Maintain two page tables during life of a transaction:
  - Current page and
  - Shadow page table.
- When transaction starts, two pages are the same.
- Shadow page table is never changed thereafter and is used to restore database in event of failure.
- During transaction, current page table records all updates to database.
- When transaction completes, current page table becomes shadow page table.
- No log record management
- However, it has an overhead of managing pages i.e. page replacement issues have to be handled.

# Example of


Current directory
(after updating
pages 2, 5)
Database disk
blocks (pages)
Shadow directory
(not updated)
1
2
3
4
5
6
Page 5 (old)
Page 1
Page 4
Page 2 (old)
Page 3
Page 6
Page 2 (new)
Page 5 (new)
1
2
3
4
5
6

# 4.6 Recovery in Multidatabase Systems

- Two-level recovery mechanism
- Global recovery manager (coordinator) needed to maintain recovery information
- Coordinator follows two-phase commit protocol
  - Phase 1: Prepare for commit message
    - Ready to commit or cannot commit signal returned
  - Phase 2: Issue commit signal
- Either all participating databases commit the effect of the transaction or none of them do

# Recovery in Multidatabase Systems (cont'd.)

- Always possible to recover to a state where either the transaction is committed or it is rolled back
- Failure during phase 1 requires rollback
- Failure during phase 2 means successful transaction can recover and commit

# 4.7 Database Backup and Recovery from Catastrophic Failures

- Database backup
  - Entire database and log periodically copied onto inexpensive storage medium
  - Latest backup copy can be reloaded from disk in case of catastrophic failure
- Backups often moved to physically separate locations
  - Subterranean storage vaults

# Database Backup and Recovery from Catastrophic Failures (cont'd.)

- Backup system log at more frequent intervals and copy to magnetic tape
  - System log smaller than database
    - Can be backed up more frequently
  - Benefit: users do not lose all transactions since last database backup

# The ARIES Recovery Algorithm

- It is designed to work with a steal , no- force approach
- It is based on: three principles
  - **WAL** (Write Ahead Logging)
    - Any change to the database object is first recorded in the log
    - The record on the log must first be save on the disk and then
    - The database object is written to disk
  - **Repeating history during redo**:
    - ARIES will retrace all actions of the database system prior to the crash to reconstruct the database state when the crash occurred.
    - Transaction that were un committed at the time of crash are undone
  - **Logging changes during undo**:
    - It will prevent ARIES from repeating the completed undo operations if a failure occurs during recovery, which causes a restart of the recovery process.

# Data Update : Four types

- **Deferred Update**:
  - All transaction updates are recorded in the local workspace (cache)
  - All modified data items in the cache is then written after a transaction ends its execution or after a fixed number of transactions have completed their execution.
  - During commit the updates are first recorded on the log and then on the database
  - If a transaction fails before reaching its commit point undo is not needed because it didn't change the database yet
  - If a transaction fails after commit (writing on the log) but before finishing saving to the data base redoing is needed from the log

- **Immediate Update**:
  - As soon as a data item is modified in cache, the disk copy is updated.
  - These update are first recorded *on the log and on the disk* by force writing, before the database is updated
  - If a transaction fails after recording some change to the database but before reaching its commit point , this will be rolled back
- **Shadow update**:
  - The modified version of a data item does not overwrite its disk copy but is written at a separate disk location.
  - Multiple version of the same data item can be maintained
  - Thus the old value ( before image BFIM) and the new value (AFIM) are kept in the disk
  - No need of Log for recovery
- **In-place update**: The disk version of the data item is overwritten by the cache version.

- Information for ARIES to accomplish recovery procedures includes:
  - Log
  - Transaction Table &
  - Dirty page table

### i. The Log and Log Sequence Number (LSN)

- The log is a history of actions executed by the DBMS in terms of a file of records stored in disk for recovery purpose
- A unique LSN is associated with every log record
  - LSN increases monotonically and indicates the disk address of the log record it is associated with.
  - In addition, each data page stores the LSN of the latest log record corresponding to a change for that page.

- A log record is written for each of the following actions:
  - updating a page
    - After modifying the page , an update record is appended to the log tail.
    - The LSN of the page is then set to the LSN of the update log record
  - commit- when a transaction decides to commit , it force writes a commit type log record containing the transaction id
  - Abort- when a transaction is aborted , an abort type log containing the transaction id is appended to the log
  - End-when a transaction is aborted or committed , an end type log record containing transaction id is appended to the log
  - undo update- when transaction is roll back its updates are undone

- A log record stores
  - the previous LSN of that transaction
  - the transaction ID
  - the type of log record.
  - For a write operation the following additional information is logged:
    - Page ID for the page that includes the item
    - Length of the updated item
    - Its offset from the beginning of the page
    - BFIM of the item

ii. The **Transaction table** and the **Dirty Page table**

- For efficient recovery following tables are also stored in the log **during checkpointing:**
  - **Transaction table**: Contains an entry for each active transaction, with information such as transaction ID, transaction status( in progresses, committed or aborted) and the LSN of the most recent log record for the transaction.
  - **Dirty Page table:** Contains an entry for each dirty page in the buffer, which includes the page ID and the LSN corresponding to the earliest update to that page.

# END